Lab 2: Integer Arithmetic

Student Name	Qimin Zhu	Student ID	6888143

Objectives:

- Get familiar with the basic MIPS integer arithmetic and logic instructions.
- Use the integer addition and subtraction, bitwise logic, and shift instructions.
- Learn some useful applications of these instructions.

Outline:

- Read through the **MIPS Lab 2 - Integer Arithmetic** sheet (PDF).
  - Read the Code Answer Template (Page 2; below).
    Each code answer should follow this template.
  - Answer questions Q2.1 – Q2.7 -- In-Lab Tasks from Section 3.5.

How to Complete This Worksheet:

- Complete this worksheet individually and submit your own copy.
- Write and test each answer in **MARS** first.
  - Assemble (F3)
  - Run (F5)
  - Check the output in the Run I/O window.
- Prepare each answer using the **Code Answer Template (page 2)**, then copy-paste your final working code into the grey answer box for each question.
- **Comment every code line** in your own words, very briefly, describing what that line does.
- Your code answers should be **complete and runnable in MARS.**
  - **Grading scores are given to assembling code that runs in MARS.**
  - **I**nclude the header comments,
  - `.data` segments
  - `.text` segments,
  - comments to show your understanding to get a score, if there's a problem assembling your code.

Code Answer Template

Answer each Lab question as a complete MIPS program in the format below.

Steps:

1. Open the **MARS editor tab.**
    a. Fill in the **header information** lines:
        i. Title,
        ii. Author (your name and student ID),
        iii. Date,
        iv. Description
        v. Input / Output of a test run.
    b. Declare your strings and variables in the `.data` segment
    c. Write your instructions in the `.text` segment after `main:`.
    d. **Exit your program** with system call: e.g.
        i. `li $v0, 10`
        ii. `syscall`
2. Make sure your code assembles, runs, and answers the question.
3. **Comment each line of code** with `#`, in your own words, briefly, e.g.
    a. `move $t0, $v0  # Copies $v0's value into register $t0.`
4. **Paste your working code into the answer box.**
    a. We grade according to whether you answered the question, and your code assembles and runs in MARS.

```
# Title:       [Program title, e.g. Echo an Integer]
# Author:      [Your Name] [Your Student ID]
# Date:        [DD/MM/YYYY]
# Description: [One sentence: what does the program do?]
# Input:       [The test input you used, e.g. 20]
# Output:      [The exact output MARS printed, e.g.]
#   Enter an integer: 20
#   You entered: 20
################## Data segment ##################
.data
prompt:  .asciiz "Enter an integer: "         # Your strings / variables here
result:  .asciiz "You entered: "
################## Code segment ##################
.text
.globl main                  # mark the “main” function as globally visible.

main:                        # main function entry
    li   $v0, 4              # Service 4 = print string
    la   $a0, prompt         # Load the address of the prompt text
    syscall                  # Print the prompt
    li   $v0, 5              # Service 5 = read integer
    syscall                  # Read the integer into $v0
    move $t0, $v0            # Copy the input value into $t0
    li   $v0, 4              # Service 4 = print string
    la   $a0, result         # Load the address of the result text
    syscall                  # Print the result text
    li   $v0, 1              # Service 1 = print integer
    move $a0, $t0            # Put the input value in $a0
    syscall                  # Print the value
    # Your code may continue here ...
    # Always end your program with the exit system call ...
```

Part 1 - Lab Tasks (Lab Sheet Section 3.5)

Answer every question using the *Code Answer Template*.

Each answer box should contain one complete, runnable, commented MIPS program.

Question Q2.1

Question / Instruction

Q2.1

Write a program that asks the user to enter two integers **A** and **B**, and then displays the result of computing the expression: `A + 2B - 5`.

Hint:

- Build the expression one step at a time:

Comment your code, complete the code template format given.

Paste your complete MIPS code answer here

Comment your code, complete the code template format given.

```
# Title:       Question 2.1
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Calculate A + 2B - 5
# Input:       Two integers A and B
# Output:      The result of A + 2B - 5

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer A: "
promptB:  .asciiz "Enter an integer B: "

################### Code segment ###################

.text

.globl main

main:

   # Print prompt A
   li   $v0, 4
   la   $a0, promptA
   syscall

   # Read integer A
   li   $v0, 5
   syscall
   move $t0, $v0

   # Print prompt B
   li   $v0, 4
   la   $a0, promptB
   syscall

   # Read integer B
   li   $v0, 5
   syscall
   move $t1, $v0

   # Calculate A + 2B - 5
   add  $t2, $t1, $t1
   sub  $t2, $t2, 5
   add  $t3, $t0, $t2

   # Print result
   li   $v0, 1
   move $a0, $t3
   syscall
```

Question Q2.2

Question / Instruction

Q2.2

Assume that **`$s1 = 0x12345678`** and **`$s2 = 0xffff9a00`**. Determine the content of registers **`$s3`** to **`$s6`** after executing the instructions listed below.

Write a program to execute these instructions and verify the content of registers **`$s3`** to **`$s6`**.

Record your predicted values in the written-answers table, then paste your verification program in the answer box.

```
and $s3, $s1, $s2
or  $s4, $s1, $s2
xor $s5, $s1, $s2
nor $s6, $s1, $s2
```

Hint:

- Calculate the four values on paper before you write the program (write answers below).
  - Work one hexadecimal digit at a time: a hex digit is 4 bits, and each operation applies to each digit on its own.
  - Service **34** prints an integer as hexadecimal, so no conversion loop is needed.

Q2.2 Written answers:

a. `and $s3, $s1, $s2` (in hexadecimal):	*$s3 = 0x1233f078*
b. `or  $s4, $s1, $s2` (in hexadecimal):	*$s4 = 0xffffde78*
c. `xor $s5, $s1, $s2` (in hexadecimal):	*$s5 = 0xedcbcc78*
d. `nor $s6, $s1, $s2` (in hexadecimal):	*$s6 = 0x00002187*

```
Paste your complete MIPS code answer here
# Title:       Question 2.2
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Perform addition, OR, XOR, and NOR operations on two hexadecimal integers
# Input:       A = 0x12345678, B = 0xFFFF9A00
# Output:      Results of A + B, A OR B, A XOR B, and A NOR B

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer A: "
promptB:  .asciiz "Enter an integer B: "
newline:  .asciiz "\n"

################### Code segment ###################

.text

.globl main

main:

   # Load integer A
   li   $s1, 0x12345678

   # Load integer B
   li   $s2, 0xFFFF9A00

   # Calculate addition
   add  $s3, $s1, $s2

   # Print result
   li   $v0, 34
   move $a0, $s3
   syscall

   # Print newline
   li   $v0, 4
   la   $a0, newline
   syscall

   # Calculate OR
   or   $s4, $s1, $s2

   # Print result
   li   $v0, 34
   move $a0, $s4
   syscall

   # Print newline
   li   $v0, 4
   la   $a0, newline
   syscall

   # Calculate XOR
   xor  $s5, $s1, $s2

   # Print result
   li   $v0, 34
```

```
move $a0, $s5
syscall

# Print newline
li   $v0, 4
la   $a0, newline
syscall

# Calculate NOR
nor  $s6, $s1, $s2

# Print result
li   $v0, 34
move $a0, $s6
syscall
```

Question Q2.3

Question / Instruction

Q2.3

Assume that **`$s1 = 0x87654321`**.

Determine the content of registers **`$s2`** to **`$s4`** after executing the instructions listed below.

Write a program to execute these instructions and verify the content of registers **`$s2`** to **`$s4`**.

Record your predicted values in the written-answers table, then paste your verification program in the answer box.

```
sll $s2, $s1, 16
srl $s3, $s1, 8
sra $s4, $s1, 12
```

Hint:

- The difference between **`srl`** and **`sra`** is the key to this question.
  - **`srl`** shifts zeros in at the top, while **`sra`** copies the sign bit in. Note that **`0x87654321`** already has its top bit set (8 = 1000 in 4-bits).
  - Shifting by a multiple of 4 moves the pattern by a whole hexadecimal digit, so these can be worked out on paper.

Comment your code, complete the code template format given.

Paste your complete MIPS code answer here

```
# Title:       Question 2.3
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Perform SLL, SRL, and SRA shift operations on a hexadecimal integer
# Input:       A = 0x87654321
# Output:      Results of SLL by 16, SRL by 8, and SRA by 12

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer A: "
promptB:  .asciiz "Enter an integer B: "
newline:  .asciiz "\n"

################### Code segment ###################

.text

.globl main

main:

   # Load integer A
   li   $s1, 0x87654321

   # Calculate SLL
   sll  $s2, $s1, 16

   # Print result
   li   $v0, 34
   move $a0, $s2
   syscall

   # Print newline
   li   $v0, 4
   la   $a0, newline
   syscall

   # Calculate SRL
   srl  $s3, $s1, 8

   # Print result
   li   $v0, 34
   move $a0, $s3
   syscall

   # Print newline
   li   $v0, 4
   la   $a0, newline
   syscall

   # Calculate SRA
   sra  $s4, $s1, 12

   # Print result
   li   $v0, 34
   move $a0, $s4
   syscall
```

Q2.3 Written answers:

a. `sll $s2, $s1, 16` (in hexadecimal):	*$s2 = 0x43210000*
b. `srl $s3, $s1, 8` (in hexadecimal):	*$s3 = 0x00876543*
c. `sra $s4, $s1, 12` (in hexadecimal):	*$s4 = 0xfff87654*

Question Q2.4

Question / Instruction

Q2.4

Write a program that asks the user to enter an alphabetic character (either lower or upper case), changes the case of the character from lower to upper and from upper to lower, and displays it.

Hint:

- Compare `'a'` (0x61) with `'A'` (0x41): they differ by 0x20, which is a single bit.
  - One logic instruction can flip that bit in both directions at once.
- Read a character with service `12` and print one with service `11`; both work on a character code in a register.
- Syscall Documentation: https://dpetersanderson.github.io/Help/SyscallHelp.html

Comment your code, complete the code template format given.

Paste your complete MIPS code answer here

```
# Title:       Question 2.4
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Toggle the case of an input letter using XOR
# Input:       One letter
# Output:      The input letter with its uppercase/lowercase case toggled

################### Data segment ###################

.data

promptA:  .asciiz "Enter a letter: "
newline:  .asciiz "\n"
space:    .asciiz " to "

################### Code segment ###################

.text

.globl main

main:

   # Print the prompt
   li   $v0, 4
   la   $a0, promptA
   syscall

   # Read a letter from input
   li   $v0, 12
   syscall
   move $s1, $v0

   # Load the case-toggle mask
   li   $s0, 0x20

   # Toggle the letter case using XOR
   xor  $s2, $s1, $s0

   # Print separator
   li   $v0, 4
   la   $a0, space
   syscall

   # Print the result
   li   $v0, 11
   move $a0, $s2
   syscall
```

Question Q2.5

Question / Instruction

Q2.5

Write a program that asks the user to **enter an integer number and reads it**.

Then ask the user to **enter** a **bit position** (between `0` and `31`) and **display the value of that bit**.

Hint:

- Bring the wanted bit down to position 0, then mask off everything else.
  - The position comes from the user, so the shift amount is in a register.
    - Use `srlv`, (not `srl`), which allows you to shift by a register (variable) amount (from the user's input).
    - `andi` with 1 then keeps just the lowest bit (of a number).

Comment your code, complete the code template format given.

Paste your complete MIPS code answer here

```
# Title:       Question 2.5
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Read an integer and display the value of a specified bit
# Input:       An integer and a bit position (0-31)
# Output:      The value of the specified bit (0 or 1)

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer: "
promptB:  .asciiz "Enter a num (0-31): "
newline:  .asciiz "\n"


################### Code segment ###################

.text

.globl main

main:

   # Print prompt for the integer
   li   $v0, 4
   la   $a0, promptA
   syscall

   # Read an integer
   li   $v0, 5
   syscall
   move $s0, $v0

   # Print prompt for the bit position
   li   $v0, 4
   la   $a0, promptB
   syscall

   # Read the bit position
   li   $v0, 5
   syscall
   move $s1, $v0

   # Shift the wanted bit to position 0
   srlv $s2, $s0, $s1

   # Keep only the lowest bit
   andi $s2, $s2, 1

   # Print the result
   li   $v0, 1
   move $a0, $s2
   syscall
```

Question Q2.6

Question / Instruction

Q2.6

Write a program that asks the user to enter a signed number and reads it.

Then display the result of **multiplying this number by `24.5`**.

Hint:

- For 24.5, you will need to make use of
  - multiplication's distributivity (as mentioned in the Lab PDF) using shifts.
  - `sra` to shift 1 place to the right to calculate x/2 (which keeps the sign).

Comment your code, complete the code template format given.

```
Paste your complete MIPS code answer here
# Title:       Question 2.6
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Multiply a signed integer by 24.5 using shifts
# Input:       A signed integer
# Output:      The integer result of 24.5 × input

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer: "
newline:  .asciiz "\n"


################### Code segment ###################

.text

.globl main

main:

   # Print prompt for the integer
   li   $v0, 4
   la   $a0, promptA
   syscall

   # Read a signed integer
   li   $v0, 5
   syscall
   move $s0, $v0

   # Calculate x × 16
   sll  $s1, $s0, 4

   # Calculate x × 8
   sll  $s2, $s0, 3

   # Calculate x / 2
   sra  $s3, $s0, 1

   # Calculate 16x + 8x
   add  $s4, $s1, $s2

   # Calculate 24x + x/2
   add  $s5, $s4, $s3

   # Print the result
   li   $v0, 1
   move $a0, $s5
   syscall
```

Question Q2.7

Question / Instruction

Q2.7
w
Write a program that asks the user to enter an unsigned number and reads it.

- Then **swap the bits at odd positions with** those **at even positions** and display the resulting number.

Hints:

- User enters 9 → Output 6
  - `1001 → 0110`
- Take the approach of mask, shift, mask, shift, combine, no need for looping or branching.
- Split the number into two halves, move each half, then join them back together.
  - **Selecting the even positions (bits 0, 2, 4, …):**
    - You need a value that has a 1 in every even position and a 0 elsewhere.
    - Write that pattern out in binary, four bits at a time — …0101 0101 0101 — and convert each group of four to one hex digit.
    - and your number with it.
  - **Selecting the odd positions (bits 1, 3, 5, …):**
    - This is the complement of the even pattern.

Comment your code, complete the code template format given.

Paste your complete MIPS code answer here

```
# Title:       Question 2.7
# Author:      Qimin Zhu 6888143
# Date:        12/09/2026
# Description: Swap the bits at odd and even positions
# Input:       An unsigned integer
# Output:      The integer with odd and even bits swapped

################### Data segment ###################

.data

promptA:  .asciiz "Enter an integer: "
newline:  .asciiz "\n"


################### Code segment ###################

.text

.globl main

main:

   # Print prompt for the integer
   li   $v0, 4
   la   $a0, promptA
   syscall

   # Read an integer
   li   $v0, 5
   syscall
   move $s0, $v0

   # Load masks
   li   $s1, 0x55555555    # Mask for even positions
   li   $s2, 0xAAAAAAAA    # Mask for odd positions

   # Extract even-position bits and shift them to odd positions
   and  $s3, $s0, $s1
   sll  $s3, $s3, 1

   # Extract odd-position bits and shift them to even positions
   and  $s4, $s0, $s2
   srl  $s4, $s4, 1

   # Combine the shifted bits
   or   $s5, $s3, $s4

   # Print the result
   li   $v0, 1
   move $a0, $s5
   syscall
```

Congratulations!

You have used the MIPS integer arithmetic, bitwise logic and shift instructions, predicted register contents and verified them in MARS, changed the case of a character with logic instructions, extracted a single bit, and multiplied using shifts and additions.

Save this worksheet as PDF, add your Student ID into the filename and upload it to the Assignment page on MyCourses.